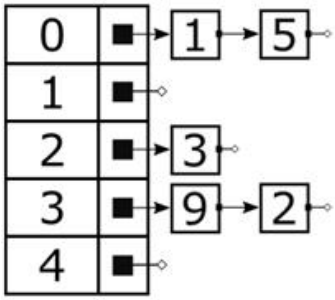


Figure C.11 Example of a hash table using chaining and a linked list for conflict resolution.

- *Open addressing*: We store elements directly in the array, and on conflicts, we generate (deterministically) another hash value for the next element to try. Hash functions for open addressing looks like

$$h(k,i) = (h'(k) + f(i,k))\%M$$

where `i` counts the number of positions already examined, and `f` is a function of the attempt and possibly the key. Open addressing allows to save memory, because there are no secondary data structures, but has some problems that makes it rarely the best choice. First and foremost, deleting elements becomes too complicated, and being its size limited on creation, this means that the table fills up quickly and needs to be reallocated. Even worse, elements tend to group in clusters and, when the table gets populated, many attempts are needed to find a free slot - basically there is a high probability that a linear number of attempts will be needed already when the table is nearly half-full.

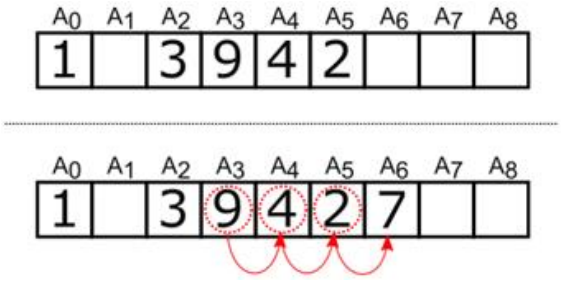


Figure C.12 A hash table using open addressing for conflict resolution. The assignment of the elements depends on the order of insertion. Assume that `f(i) = i` and that we have the same collisions as in figure C.11.